プロジェクトポートフォリオ

武藤雅裕 — AI エンジニア・ロボティクス研究者

最終更新：2026年11月10日

- ポートフォリオサイト: huggingface.co/spaces/mutomasa/engineering-research-portfolio
- GitHub: github.com/mutomasa
- Hugging Face: huggingface.co/mutomasa
- Speaker Deck: speakerdeck.com/mutomasa

対応できる領域

領域	主な技術	代表プロジェクト
LLM・AI Agent	vLLM、Qwen、LangGraph、CrewAI、MCP、LLM as a Judge、Lean 4 による検証	Jev Decision Router、Lean Agent Verification Lab
機械学習・生成モデル	埋め込み評価、拡散モデル、Transformer の実装、量子機械学習	EmbeddingGemma Multimodal Lab
グラフ・知識グラフ・RAG	Neo4j、Cypher、Graph RAG、GNN（PyTorch Geometric）	Security Attack Graph 3D
コンピュータビジョン	物体検出・追跡（YOLO、DETR、OWL-ViT、DeepSORT）、セグメンテーション、行動認識	Grounding SAM 2、DETR 可視化
データ基盤・インフラ	Snowflake、dbt、Cloudflare（Workers・R2・Iceberg）、Kafka、Go	Cloudflare Basin Iceberg、Kafka 3D ストリーミング可視化
ロボティクス・空間AI・3D	3D Gaussian Splatting、SAM 3.1、ROS 2、強化学習（PPO）、Three.js	SpatialSAM Studio、Smart 3D Ecosystem

仕事の進め方の特徴

- **ローカル GPU（8GB）で完結させる設計。** 推論も学習も手元の GPU で動かし、外部の有料 API に頼らない構成を多く作っています。機密データを外に出せない案件に向きます。
- **数値で確かめる。** デモで終わらせず、評価セット・評価指標・実測値（IoU、nDCG、レイテンシなど）を README に残します。
- **壊れても止まらない設計。** LLM や推論サーバーが落ちたときにルールベースへ戻る仕組みや、LLM に危険な操作をさせないガードレールを組み込みます。
- **設計書から始める。** 設計書・TODO・ADR を先に書き、AI Coding Agent（Claude Code など）と小さな単位でテストしながら実装します。

プロジェクト一覧

1 ページに 1 プロジェクトを載せています。各ページには、構成図（Mermaid）、成果物の概要、工夫している点、GitHub へのリンクを書いています。各リポジトリの README と設計書を読んで書きました。

中断 の印があるものは、開発を止めているプロジェクトです。

#	プロジェクト	カテゴリ	状態
1	SpatialSAM Studio（gaussian-sam-studio）	ロボティクス・空間AI・3D	実装済み
2	Smart 3D Ecosystem — Boids + 機械学習	ロボティクス・空間AI・3D	実装済み
3	gsplat Gaussian Splatting 比較デモ（3DGS）	ロボティクス・空間AI・3D	実装済み
4	量子ロボット経路探索（quantum_policy_gradient）	ロボティクス・空間AI・3D	実装済み
5	Grounding SAM 2 動画物体追跡	コンピュータビジョン	実装済み
6	DETR 可視化アプリ	コンピュータビジョン	実装済み
7	SegFormer セマンティックセグメンテーション	コンピュータビジョン	実装済み
8	OWL-ViT オープンボキャブラリ物体検出	コンピュータビジョン	実装済み
9	GLIP 物体検出・グラウンディング	コンピュータビジョン	実装済み
10	Structure-CLIP + OWL-ViT	コンピュータビジョン	実装済み
11	DeepSORT オブジェクトトラッキング	コンピュータビジョン	実装済み
12	YOLOv8 物体検出・可視化	コンピュータビジョン	実装済み
13	画像シーングラフ生成（Graph R-CNN 風）	コンピュータビジョン	実装済み
14	MMAction2 行動認識デモ	コンピュータビジョン	実装済み
15	PySlowFast 行動認識（最小構成）	コンピュータビジョン	実装済み
16	ViTMAE 画像再構築	コンピュータビジョン	実装済み
17	SimCLR 対照学習の可視化	コンピュータビジョン	実装済み
18	画像キャプション評価アプリ（Qwen-VL）	コンピュータビジョン	実装済み
19	画像処理ツールキット（manipulate_image_lists）	コンピュータビジョン	実装済み
20	Jev Decision Router デモ	LLM・AI Agent	実装済み
21	Lean Agent Verification Lab	LLM・AI Agent	実装済み
22	Phoenix LLM Observability デモ	LLM・AI Agent	実装済み
23	Prompt Optimizer（LangChain 対応）	LLM・AI Agent	実装済み
24	Agent Evolution（Clojure）	LLM・AI Agent	実装済み
25	Research OS	LLM・AI Agent	実装済み
26	JAEA 特許・報告書検索（ローカル RAG）	LLM・AI Agent	実装済み
27	DLT スライド生成システム	LLM・AI Agent	実装済み
28	vLLM サーバーとモデルのダウンロード	LLM・AI Agent	実装済み
29	RoboPaper TUI（論文収集エージェント）	LLM・AI Agent	中断
30	Security Attack Graph 3D	グラフ・知識グラフ・RAG	実装済み
31	Graph RAG Video（動画分析）	グラフ・知識グラフ・RAG	実装済み
32	Graph RAG アプリ	グラフ・知識グラフ・RAG	実装済み
33	日本語テキストからの知識グラフ構築	グラフ・知識グラフ・RAG	実装済み
34	GNN 可視化アプリ（R-GCN on Cora）	グラフ・知識グラフ・RAG	実装済み
35	Node2Vec + GCN 可視化	グラフ・知識グラフ・RAG	実装済み
36	ネットワーク指標の可視化	グラフ・知識グラフ・RAG	実装済み

#	プロジェクト	カテゴリ	状態
37	EmbeddingGemma Multimodal Lab	機械学習・生成モデル	実装済み
38	Diffusion Transformer vs U-Net	機械学習・生成モデル	実装済み
39	AI 駆動創薬 × 量子コンピュータ デモ	機械学習・生成モデル	実装済み
40	Simple GPT（学習用）	機械学習・生成モデル	実装済み
41	BERT + UMAP + HDBSCAN テキストクラスタリング	機械学習・生成モデル	実装済み
42	CLAP 音声とテキストの理解	機械学習・生成モデル	実装済み
43	Whisper 日本語音声認識デモ	機械学習・生成モデル	実装済み
44	PCA と NMF の可視化	機械学習・生成モデル	実装済み
45	ランダムフォレストの決定境界	機械学習・生成モデル	実装済み
46	Cloudflare Basin + Iceberg	データ基盤・インフラ・ツール	実装済み
47	Kafka 3D ストリーミング可視化	データ基盤・インフラ・ツール	実装済み
48	リアルタイム共有ホワイトボード	データ基盤・インフラ・ツール	実装済み
49	Raft の可視化（Go）	データ基盤・インフラ・ツール	実装済み
50	Git Uncommit Checker	データ基盤・インフラ・ツール	実装済み
51	ターミナル Todo アプリ	データ基盤・インフラ・ツール	実装済み
52	Ghostty AI-Native 開発環境	データ基盤・インフラ・ツール	実装済み
53	Engineering Research Portfolio（このサイト）	データ基盤・インフラ・ツール	実装済み

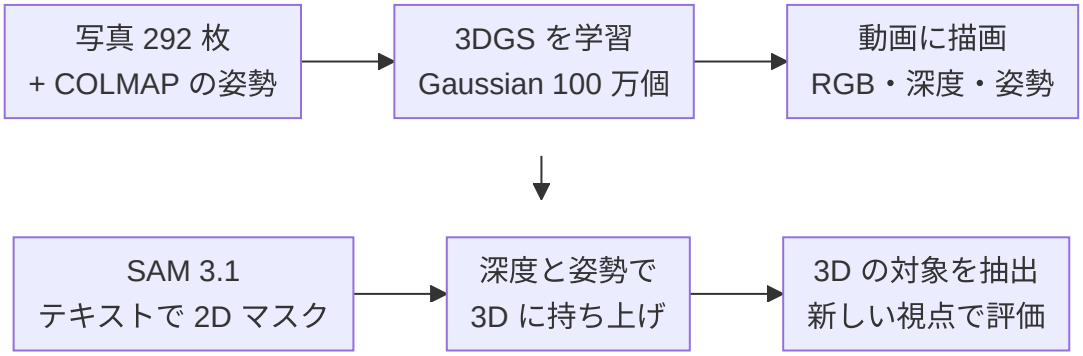
1. SpatialSAM Studio（gaussian-sam-studio）

SAM 3.1 | 3D Gaussian Splatting | gsplat | Streamlit | MLflow

GitHub: github.com/mutomasa/gaussian-sam-studio

成果物の概要

テキストで「何を切り出すか」を指定して、動画と 3D シーン（3D Gaussian Splatting）をセグメンテーションする Streamlit アプリです。写真 292 枚から作った 3D シーンの中の盆栽を、「bonsai」というテキストだけで 3D のまま切り出せます。推論・3DGS の学習・描画は、すべてローカル GPU（8GB）で行います。



工夫している点

- **2D のモデルで 3D を切り出す。** SAM 3.1 は 2D の画像しか扱えません。そこで、3D シーンをいったん動画に描画して SAM にかけ、その結果を深度とカメラ姿勢で各 Gaussian に戻しています。
- **学習に使っていない視点で検証する。** 切り出した 3D の対象を新しい視点で描き、その視点の実写に SAM をかけた結果と比べました。IoU は 0.908（bonsai）、0.927（table）です。
- **8GB の GPU で全工程が動く。** 3DGS の学習は 7000 step・9 分・VRAM 1.2GB で、評価用視点の PSNR は 30.7 です。動画セグメンテーションは DAVIS で J&F 0.95〜0.97 を確認しました。
- **実験を MLflow で記録する。** 条件と結果を一覧で比べられます。GPU がない環境では、ダミーのセグメンタで UI とパイプラインを確認できます。

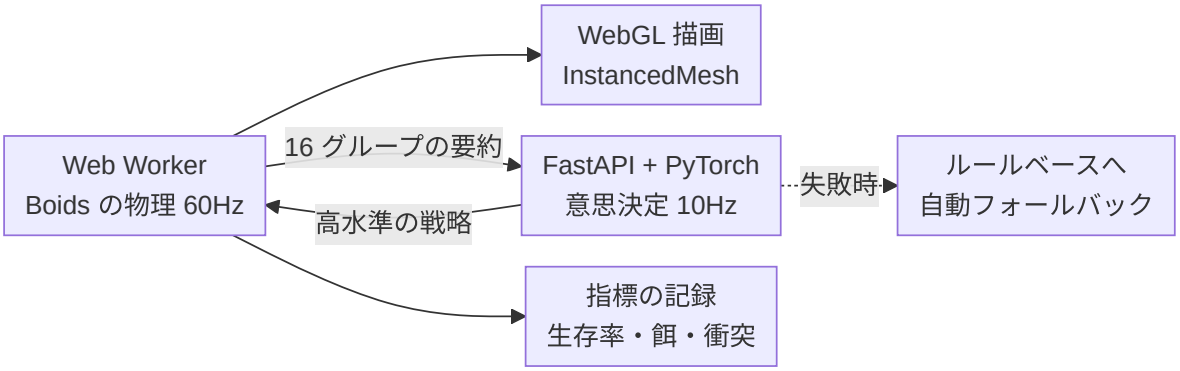
2. Smart 3D Ecosystem — Boids + 機械学習

Three.js | Web Worker | PyTorch | PPO | FastAPI | WebSocket

GitHub: github.com/mutomasa/smart-3d-ecosystem

成果物の概要

数千匹の魚が 3D の水中で群れを作り、捕食者から逃げて餌を探すデモです。群れの意思決定を、ルールベースと学習済み方策（PyTorch・PPO）で切り替えられます。同じ条件で何が違うのかを、Streamlit のダッシュボードで数値で比べられます。



工夫している点

- **高頻度のシミュレーションと ML を分けた。** 物理更新（60Hz）はブラウザの Web Worker で動かします。サーバーとやり取りするのは 10Hz で、送るのも 16 グループの要約（20 次元）だけです。帯域と遅延が破綻しない構成にしました。
- **壊れても止まらない。** 推論のタイムアウト（250ms）、切断、例外が起きると、直前の行動を 1 秒保ってからルールベースに戻ります。学習済みモデルがなければ「untrained」と表示し、学習済みと偽りません。
- **学習と推論のずれを防ぐ。** 観測と行動の形式を 1 つの JSON スキーマにまとめ、学習と推論で同じ前処理コードを使います。TypeScript と Python の実装が一致することもテストしています。
- **再現できる比較。** シードを固定し、学習に使っていないシードで評価します。学習済み方策がルールに勝てていない結果もそのまま見せ、「ML を入れる価値があるか」の判断材料にしています。

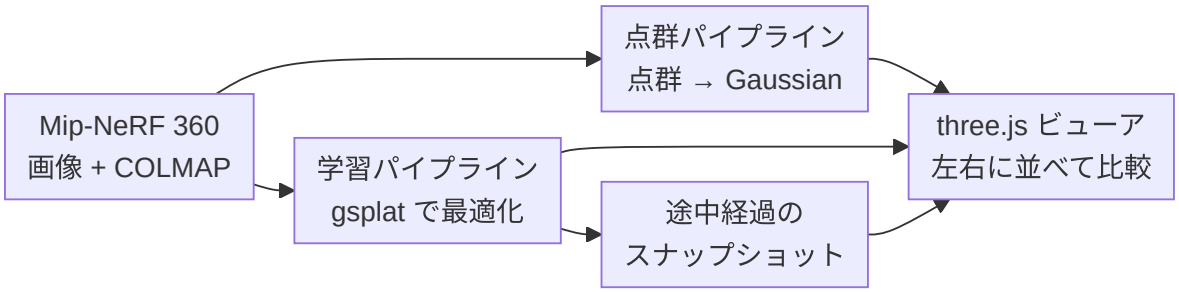
3. gsplat Gaussian Splatting 比較デモ（3DGS）

gsplat | COLMAP | FastAPI | TypeScript | three.js

GitHub: github.com/mutomasa/3DGS

成果物の概要

同じオープンデータ（Mip-NeRF 360）から 2 通りの方法で Gaussian Splat を作り、three.js のビューアで左右に並べて比べる Web アプリです。1 つは COLMAP の点群をそのまま Gaussian にする方法、もう 1 つは姿勢付き画像で gsplat を学習する方法です。



工夫している点

- **学習の途中経過を再生できる。** 指定したステップごとに中間の Gaussian を書き出し、点群が最適化されていく様子をアニメーションで見せられます。発表や勉強会で使うための機能です。
- **学習中でも追いかけて再生する。** 学習が続いていてスナップショットが増えていく間も、一覧を取り直して再生を続けます。
- **CUDA ツールチェーンがない環境でも動かす。** nvcc を仮想環境の中だけに入れて gsplat のカーネルをビルドする手順を用意しました。システムには手を入れません。
- **バックエンドとフロントエンドを分けた。** Python（uv）の CLI と FastAPI、TypeScript（Vite）のビューアを分けて構成しています。

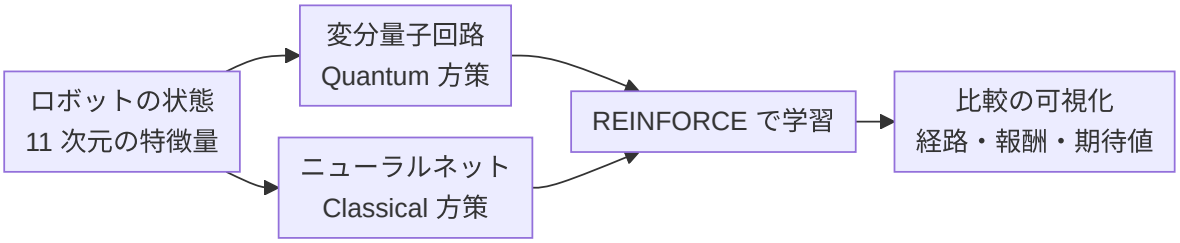
4. 量子ロボット経路探索（quantum_policy_gradient）

量子機械学習 | 変分量子回路 | REINFORCE | Streamlit

GitHub: github.com/mutomasa/quantum_policy_gradient

成果物の概要

グリッド上のロボットが、障害物と危険区域を避けてゴールへ向かう経路探索のデモです。変分量子回路（VQC）による方策勾配法と、古典ニューラルネットワークによる方策勾配法を、同じ条件で学習して比べます。



工夫している点

- **公平に比べられる。** 同じグリッド設定なら、量子と古典は常に同じレイアウトで学習します。2 つの経路を同じグリッドに重ねて表示できます。
- **量子回路の条件を変えて試せる。** 量子ビット数（2〜8）、回路の深さ、有限ショット、脱分極ノイズを UI から変えられます。NISQ（ノイズのある中規模量子計算機）の制約を体感できます。
- **結果を 6 つの観点で見せる。** 行動確率のヒートマップ、移動のアニメーション、累積報酬、各量子ビットの期待値の推移などを確認できます。
- **理論とコードの対応を説明する。** README で、パラメータシフト則などの理論と実装の対応表を示しています。

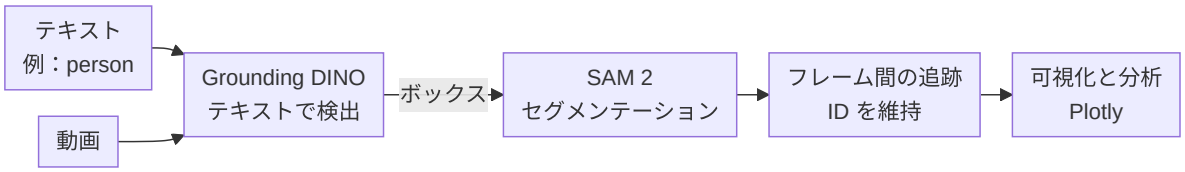
5. Grounding SAM 2 動画物体追跡

Grounding DINO | SAM 2 | Streamlit | Plotly

GitHub: github.com/mutomasa/grounding_sam2_app

成果物の概要

テキストで指定した物体を、動画の中で検出・セグメンテーション・追跡する Streamlit アプリです。動画（MP4・AVI・MOV・MKV）をアップロードし、「人」「車」のような言葉で追いかける対象を指定します。



工夫している点

- **学習なしで任意の物体を追える。** 検出に Grounding DINO を使うので、決まったクラスだけでなく、言葉で表せる物体なら追加の学習なしで検出できます。
- **検出と境界を役割分担させる。** 位置の特定は Grounding DINO、正確な境界は SAM 2 に任せ、2 つのモデルの得意な部分を組み合わせています。
- **結果を分析できる。** マスクの重ね合わせに加えて、Plotly のグラフで検出結果を分析できます。

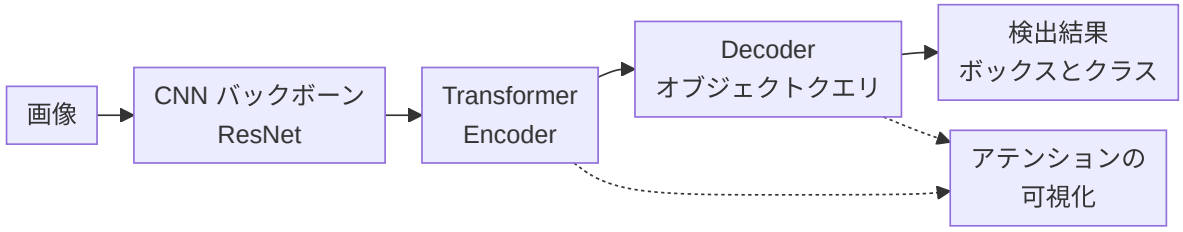
6. DETR 可視化アプリ

DETR | Transformer | PyTorch | Streamlit

GitHub: github.com/mutomasa/detr_app

成果物の概要

Transformer で物体を検出するモデル DETR について、検出結果だけでなくモデルの中で何が起きているかを見せる Streamlit アプリです。ResNet-50/101 版とパノプティックセグメンテーション版に対応しています。



工夫している点

- **中身が見える。** アテンションの重み、オブジェクトクエリの動き、隠れ状態を可視化し、DETR がどこを見て判断しているかを確認できます。
- **構造の図解つき。** DETR の構造と処理の流れを、アプリの中で図で説明しています。
- **デバッグ情報を出す。** モデルの情報、メモリ使用量などを表示し、検証に使えるようにしています。

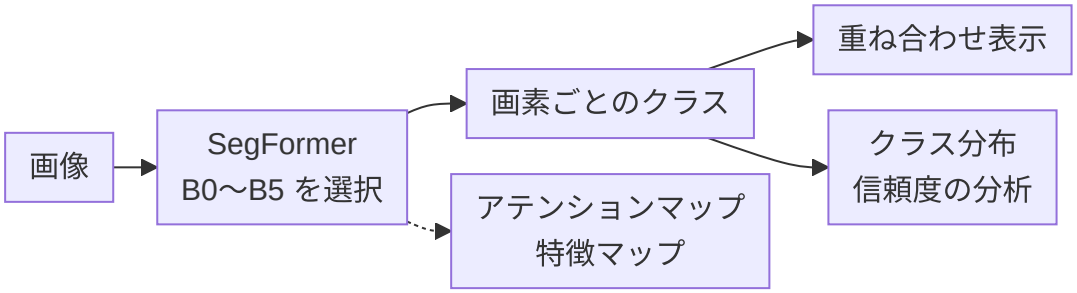
7. SegFormer セマンティックセグメンテーション

SegFormer | ADE20K | Cityscapes | Streamlit

GitHub: github.com/mutomasa/segformer_app

成果物の概要

SegFormer（B0〜B5）で画像を画素単位に分類するセマンティックセグメンテーションのアプリです。ADE20K と Cityscapes の学習済みモデルに対応し、結果に加えてモデル内部の動作も可視化します。



工夫している点

- **モデルの大きさを選べる。** B0（軽量）から B5（高精度）まで切り替えて、精度と速度の違いを確かめられます。
- **内部を分析できる。** アテンションマップと特徴マップを表示し、処理時間とメモリ使用量も計測します。
- **結果を統計で見る。** クラス別の画素数や信頼度の分布を表示し、どのクラスが不安定かを把握できます。

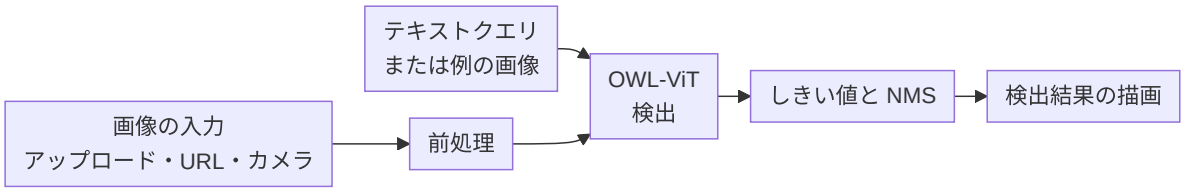
8. OWL-ViT オープンボキャブラリ物体検出

OWL-ViT | Hugging Face Transformers | Streamlit

GitHub: github.com/mutomasa/owl-vit-app

成果物の概要

OWL-ViT で、決まったクラスに限らない物体（オープンボキャブラリ）を検出する Streamlit アプリです。テキストのクエリでも、例として与えた画像でも検出できます。画像はサンプル、アップロード、URL、カメラ撮影から入力できます。



工夫している点

- **単一責任で設計した。** 画像の読み込み・検証・前処理・可視化をそれぞれ別のクラスに分け、再利用とテストをしやすくしています。
- **3 つの検出モード。** 基本の検出、テキストで指定する検出、類似画像で指定する検出を切り替えられます。
- **使いやすさ。** プリセットのクエリ、カンマ区切りの複数入力、信頼度しきい値の自動調整、日本語と英語の翻訳に対応しています。

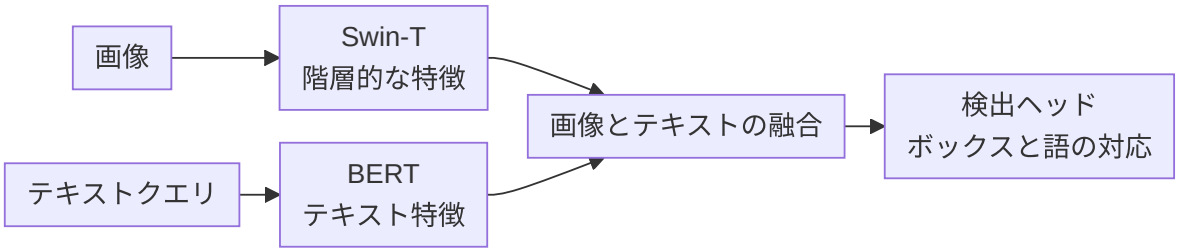
9. GLIP 物体検出・グラウンディング

GLIP | Swin Transformer | BERT | Streamlit

GitHub: github.com/mutomasa/glip_app

成果物の概要

Microsoft の GLIP（Grounded Language-Image Pre-training）で、テキストのクエリに対応する物体を画像から検出・位置特定（グラウンディング）する Streamlit アプリです。バックボーンには Swin Transformer（Swin-T）を使っています。



工夫している点

- **言葉と物体を結びつける。** 文中の語と画像中の物体を対応させるので、新しいカテゴリも追加の学習なしで検出できます。
- **アーキテクチャを解説する。** Swin Transformer のシフトウィンドウや階層構造、Swin-T の仕様（約 28M パラメータなど）を README で説明しています。

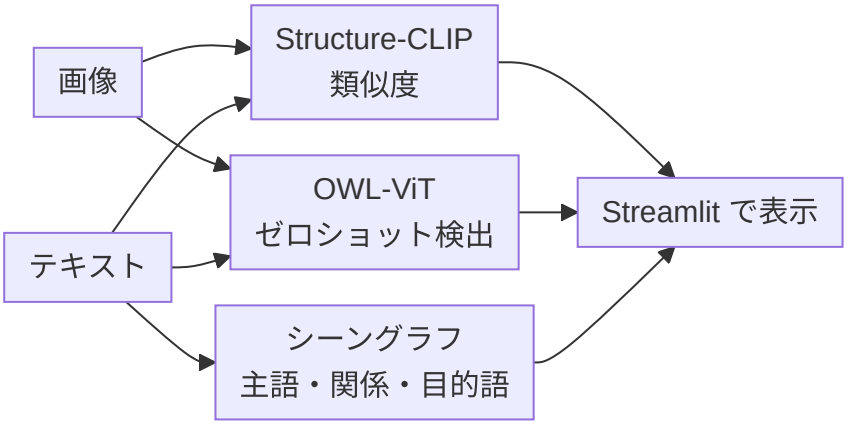
10. Structure-CLIP + OWL-ViT

Structure-CLIP | OWL-ViT | Graphviz | Streamlit

GitHub: github.com/mutomasa/structure_clip_owl_vit_app

成果物の概要

Structure-CLIP（AAAI 2024）の CLIP 互換実装によるテキストと画像の類似度計算、OWL-ViT によるゼロショット検出、主語・関係・目的語のシーングラフ可視化を 1 つにまとめた Streamlit アプリです。



工夫している点

- **論文の実装を最小構成で動かす。** 元の実装は追加の前処理や学習済みチェックポイントが必要ですが、同梱の CLIP 実装を直接使い、軽いデモとして動くようにしました。
- **構造の理解を見せる。** 文の構造（誰が・何を・どうした）をシーングラフで表示し、Structure-CLIP の狙いを直感的に伝えます。

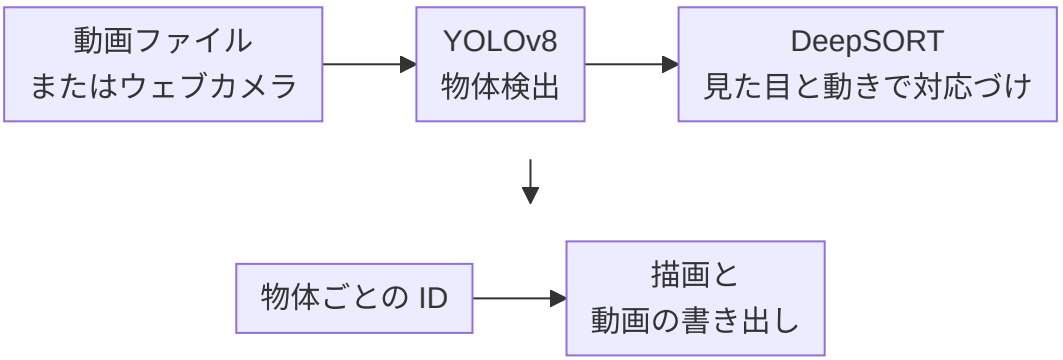
11. DeepSORT オブジェクトトラッキング

YOLOv8 DeepSORT OpenCV Streamlit

GitHub: github.com/mutomasa/deepsort_app

成果物の概要

YOLOv8 で検出した物体を DeepSORT で追跡し、動画の中の複数の物体に ID を付けて表示する Streamlit アプリです。処理済みの動画はダウンロードできます。



工夫している点

- **用途に合わせた 3 つの版。** 動画全体を処理してからダウンロードする基本版、フレームごとに結果を見るリアルタイム版、ウェブカメラ入力のライブ版を用意しました。
- **性能を見ながら使える。** リアルタイム版では FPS と処理時間を画面に表示します。

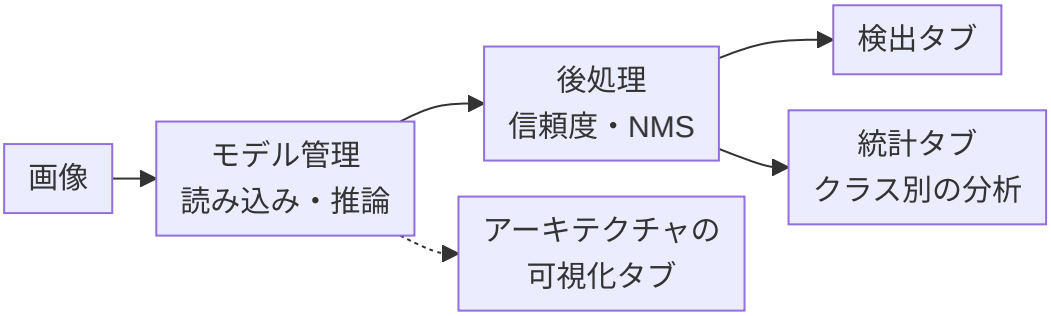
12. YOLOv8 物体検出・可視化

YOLOv8 Ultralytics OpenCV Streamlit

GitHub: github.com/mutomasa/yoro_app

成果物の概要

YOLOv8 で画像の物体（COCO の 80 クラス）を検出し、検出結果の統計と YOLOv8 のアーキテクチャ図を表示する Streamlit アプリです。



工夫している点

- **役割ごとにモジュールを分けた。** モデル管理、可視化、UI 部品を別のファイルにし、エラー処理もモデル管理側にまとめています。
- **しきい値を調整できる。** 信頼度と NMS のしきい値を変えて、検出結果の変化を確かめられます。

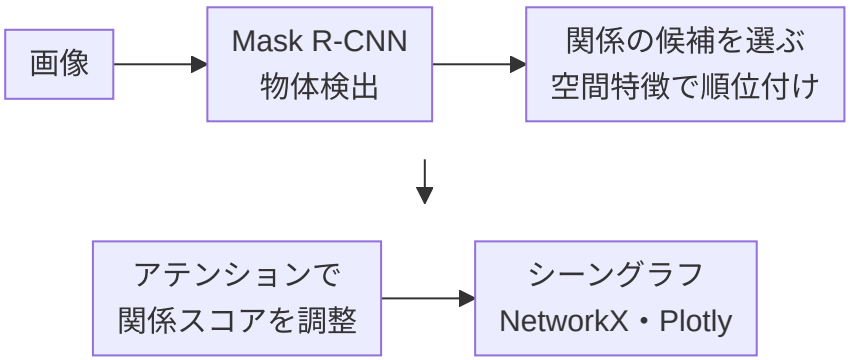
13. 画像シーングラフ生成（Graph R-CNN 風）

Mask R-CNN | Multi-Head Attention | NetworkX | Plotly

GitHub: github.com/mutomasa/scene_graph_app

成果物の概要

画像から「何が・どの物体と・どんな関係にあるか」を表すシーングラフを作る最小実装です。Graph R-CNN（ECCV 2018）を参考にしています。



工夫している点

- **論文の中心部分を自分で実装した。** 関係候補の選択（RePN）とアテンションによる関係スコアの調整を、簡易版として実装しています。
- **空間特徴で候補を絞る。** すべての物体の組み合わせについて IoU と相対位置・サイズを計算し、近いものから順に候補にします。

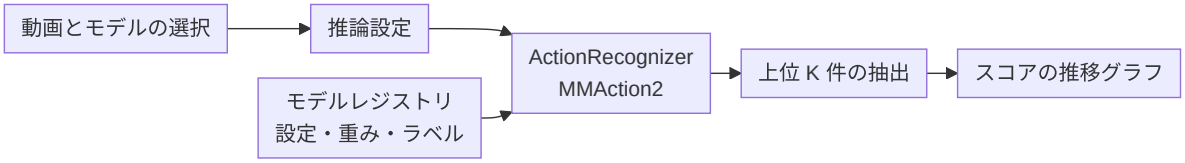
14. MMAction2 行動認識デモ

MMAction2 TSN SlowFast Streamlit

GitHub: github.com/mutomasa/mmaction1_app

成果物の概要

アップロードした動画の中の行動（歩く、跳ぶなど）を認識する Streamlit アプリです。上位のラベル、時間ごとのスコアの推移グラフ、動画のプレビューを表示します。



工夫している点

- **速さと精度でモデルを選べる。** CPU でも動く軽量な TSN と、GPU を使う高精度な SlowFast を切り替えられます。
- **層を分けた設計。** UI、推論のサービス層、モデルのレジストリ（設定・重み・ラベルの管理）を分けています。

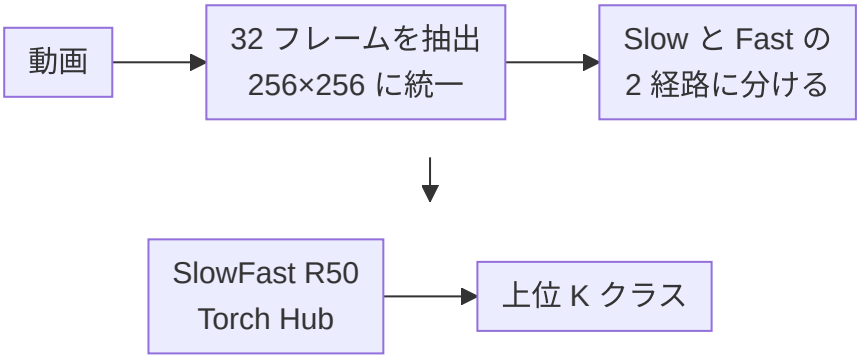
15. PySlowFast 行動認識（最小構成）

SlowFast | PyTorchVideo | Kinetics-400 | Streamlit

GitHub: github.com/mutomasa/pyslowfast_app

成果物の概要

Meta の SlowFast モデル（Kinetics-400 で学習済み）で、アップロードした動画の行動を認識する最小構成のアプリです。



工夫している点

- **前処理を明確にした。** フレームの抽出、リサイズ、2 経路のテンソル作成までを、パイプラインとして分けています。
- **ラベルがなくても動く。** 公式ラベルを取得できない環境では、仮のラベルを生成して動作を確認できます。

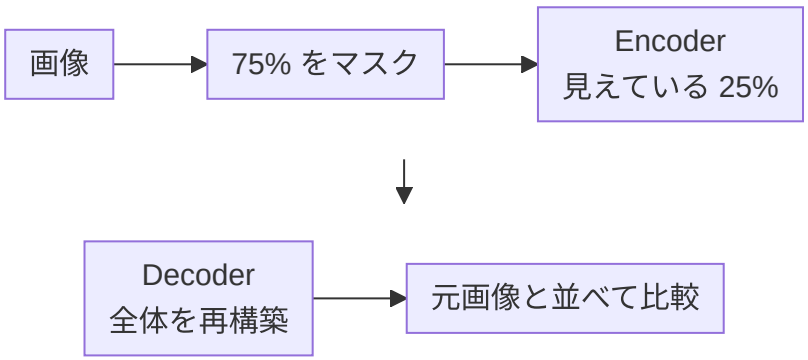
16. ViTMAE 画像再構築

ViTMAE | Vision Transformer | Streamlit

GitHub: github.com/mutomasa/vitmae

成果物の概要

ViTMAE で画像の 75% を隠し、残りの 25% から元の画像を再構築する Streamlit アプリです。元の画像、隠した画像、再構築した画像を並べて表示します。



工夫している点

- **自己教師あり学習の考え方を見せる。** どこを隠したかを可視化し、モデルが周りから全体を推測する様子を確かめられます。
- **入力と出力の扱いやすさ。** サンプル、アップロード、URL から画像を入れられ、結果は PNG で保存できます。

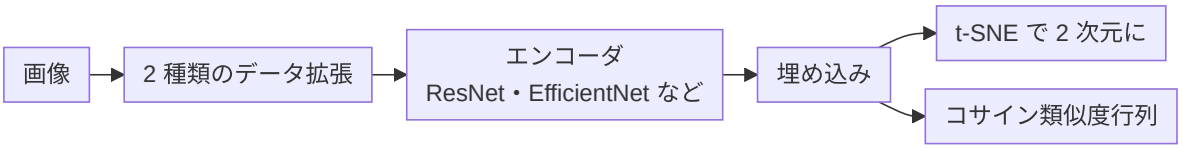
17. SimCLR 対照学習の可視化

SimCLR ResNet t-SNE Streamlit

GitHub: github.com/mutomasa/simclr_app

成果物の概要

対照学習 SimCLR の仕組みを可視化する Streamlit アプリです。データ拡張、特徴抽出、t-SNE による埋め込み空間、コサイン類似度行列を確認できます。



工夫している点

- **バックボーンを切り替えられる。** ResNet18/50、EfficientNet、MobileNet で、埋め込みの違いを比べられます。
- **学習の過程も見せる。** 学習曲線と統計情報を表示します。

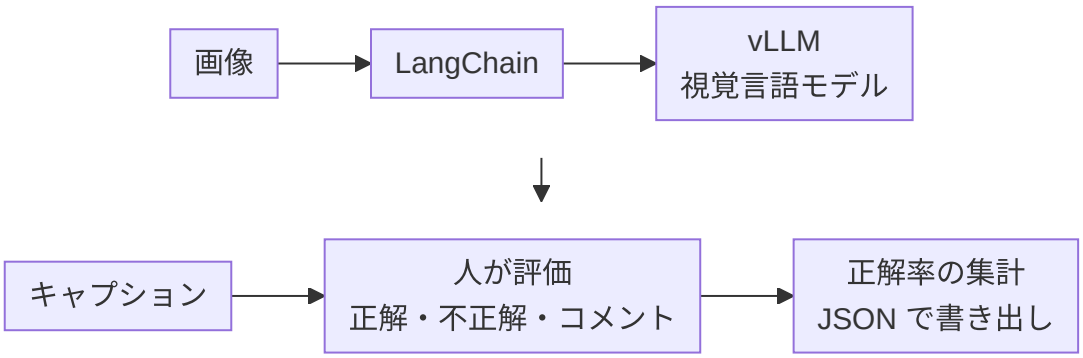
18. 画像キャプション評価アプリ（Qwen-VL）

Qwen2.5-VL | vLLM | LangChain | Streamlit

GitHub: github.com/mutomasa/qwen-vl-caption

成果物の概要

vLLM 上の視覚言語モデルで画像のキャプションを生成し、人が正解・不正解を付けて正解率を集計する評価アプリです。



工夫している点

- **人による評価を組み込んだ。** 生成して終わりではなく、評価ボタンと不正解時のコメントで、モデルの弱点を記録できます。
- **評価データを再利用できる。** 評価結果を JSON で書き出し、モデルの比較や改善に使えます。

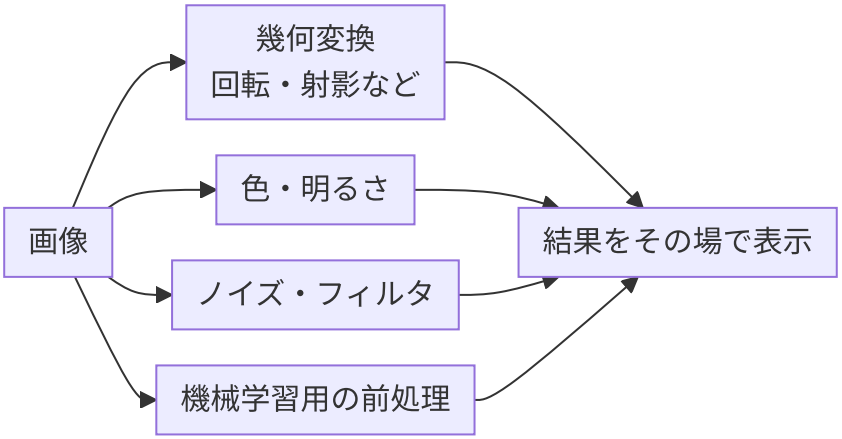
19. 画像処理ツールキット（manipulate_image_lists）

OpenCV Pillow Streamlit

GitHub: github.com/mutomasa/manipulate_image_lists

成果物の概要

画像処理の操作をカテゴリ別に整理し、パラメータを変えながら試せる Streamlit アプリです。幾何変換、色・明るさの変換、ノイズとフィルタ、機械学習用の前処理の 4 種類を扱います。



工夫している点

- **格子状のボタン配置。** 操作を格子状に並べ、結果をボタンのすぐ上に表示するので、多くの変換を素早く比べられます。
- **データ拡張の確認に使える。** 機械学習の前処理（データ拡張）が画像にどう効くかを、学習の前に目で確かめられます。

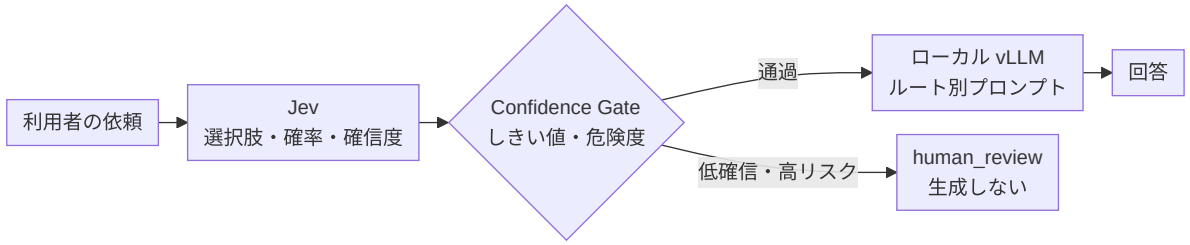
20. Jev Decision Router デモ

Jev | vLLM | Qwen2.5 | Streamlit | Agentic Mesh

GitHub: github.com/mutomasa/jev-decision-router-demo

成果物の概要

Jev を「判断のレイヤー」、ローカルの vLLM を「生成のレイヤー」として組み合わせたデモです。利用者の依頼を Jev で分類し、確信度のチェック（Confidence Gate）を通ったものだけを、ルートごとのシステムプロンプトで vLLM に渡します。Agentic Mesh の Decision Plane（判断を担う層）の最小 PoC です。



工夫している点

- **判断をコードで扱える値にした。** 判断が LLM の出力文に埋もれないよう、選択肢・確率・確信度を型付きの値として受け取ります。しきい値の制御、ログ、テストを普通のコードで書けます。
- **危険な依頼は生成の前に止める。** 「本番 DB を削除して」のような依頼は、判断の時点で人の確認に回し、LLM は呼びません。実行するかどうかはソフトウェアのポリシーが決めます。
- **判断は速い。** 判断は約 0.2〜0.27 秒で、生成（1.3〜21 秒）に比べて小さく、ルーティングを加えても応答時間はほとんど変わりません。
- **精度を測れる。** 24 件の評価セットで全件正解（Gate 適用後は 23/24）でした。評価コマンドで、設定を変えたときの精度をすぐ確かめられます。
- **部品を差し替えられる。** 判断と生成を小さなインターフェースで分け、別のモデルや Mock に替えてもルーターと UI は変わりません。

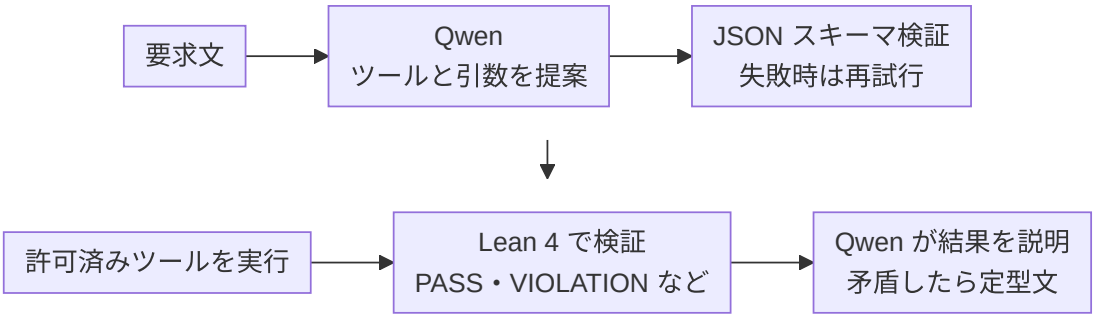
21. Lean Agent Verification Lab

Lean 4 | vLLM | Qwen3-1.7B | OR-Tools | Streamlit

GitHub: github.com/mutomasa/lean-agent-verification-lab

成果物の概要

ローカル GPU で動く AI Agent（vLLM + Qwen3-1.7B）と、手書きの Lean 4 による形式検証を組み合わせた 3 つのデモです。送金の可否、RBAC の権限表、OR-Tools による人員配置について、業務ルールを満たすことを Lean で証明します。



工夫している点

- **LLM に判定させない。** Qwen は許可済みのツールと引数を「提案するだけ」で、合否は Python と Lean が決めます。
- **LLM の出力を何重にも確かめる。** 構造化出力で形式を縛り、スキーマで検証し、要求文に現れない数値は警告します。判定と矛盾する説明は定型文に置き換えます。
- **判定の種類をはっきり分けた。** 「違反を証明した（VIOLATION）」と「証明が通らなかった（PROOF_FAILED）」を区別するなど、6 つの状態で結果を返します。
- **最適化の結果も検証する。** OR-Tools が求めた配置が全制約を満たすことを Lean で確かめます（最適性は証明しない、と範囲を明記）。

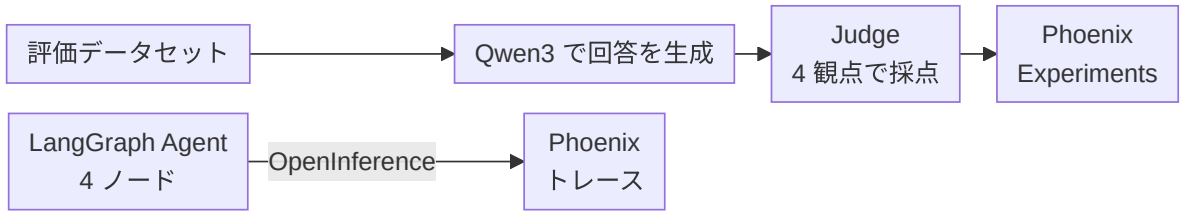
22. Phoenix LLM Observability デモ

Arize Phoenix | OpenInference | LangGraph | vLLM | Qwen3

GitHub: github.com/mutomasa/phoenix-llm-observability-demo

成果物の概要

Arize Phoenix とローカルの Qwen3（vLLM）で、LLM アプリの評価と可観測性を体験するデモです。1 つ目は、生成した回答を別の Qwen 呼び出し（Judge）が 4 つの観点で採点する LLM as a Judge です。2 つ目は、4 ノードの LangGraph Agent の実行を OpenInference で追跡します。



工夫している点

- **外部の商用 API を使わない。** 生成も採点もローカルの Qwen3 で行い、可視化は Phoenix 標準の画面だけで完結させました。
- **再現できる環境。** Phoenix は Docker、依存は uv.lock で固定し、各ライブラリの検証済みバージョンを README に記録しています。
- **8GB の GPU に合わせた運用。** vLLM はプロジェクトの依存に入れず uvx で実行し、メモリが足りないときの設定（最大長、小さいモデル）も用意しました。

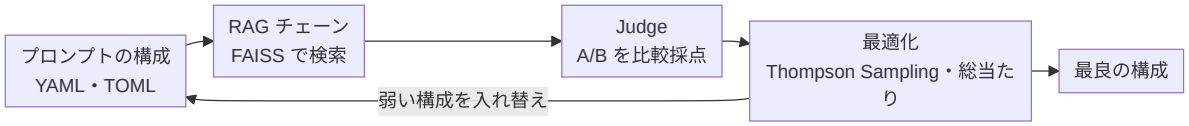
23. Prompt Optimizer（LangChain 対応）

LangChain | LangGraph | LLM as a Judge | Thompson Sampling | FAISS

GitHub: github.com/mutomasa/prompt_optimaizer

成果物の概要

プロンプトを YAML / TOML で管理し、LLM as a Judge で評価しながら最適化するフレームワークです。RAG 向けに、引用の正しさと忠実性を重視した評価ループを持ちます。



工夫している点

- **2 つの探索方法。** 全構成を総当たりで対戦させるトーナメントと、Beta 分布で有望な構成に試行を集中させる Thompson Sampling を選べます。
- **Judge の偏りを抑える。** A と B の提示順をランダムにし、出力の JSON を厳密に検証します。僅差（0.05 未満）は引き分けにします。
- **既存のパイプラインに組み込める。** 最良の設定を LangChain / LangGraph のノードとしてそのまま使えます。

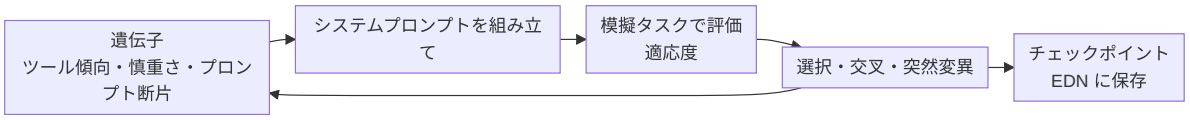
24. Agent Evolution（Clojure）

Clojure | 遺伝的アルゴリズム | LLM Agent

GitHub: github.com/mutomasa/clojure-agent-evolution

成果物の概要

LLM エージェントの振る舞いを遺伝子として表し、遺伝的アルゴリズムで世代ごとに進化させるプロトタイプです。遺伝子には、ツールの使い方の偏り、慎重さ、システムプロンプトに入れる方針の断片などを持たせます。



工夫している点

- **定期実行するだけで進化が続く。** 個体群と履歴をファイルに保存・復元するので、cron などで実行するたびに続きから進化します。
- **良い個体を失わない。** エリート保存戦略で、それまでの最良の個体を残します。
- **依存ゼロ。** Clojure の標準ライブラリだけで書き、外部ライブラリ・ネットワーク・API キーなしで動きます。テストも付けています。

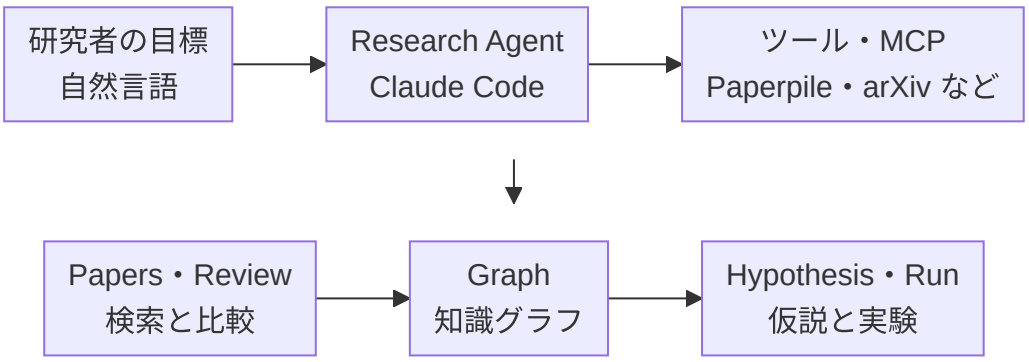
25. Research OS

Python | Textual（TUI） | Claude Code | MCP | SQLite

GitHub: github.com/mutomasa/research_os

成果物の概要

文献検索から論文レビュー、知識グラフ、仮説、実験の実行までを、1 つの TUI（ターミナルの画面）でつなぐ研究開発 IDE です。Paperpile を文献 DB、Claude Code を Research Agent、MCP を外部サービスとの接続に使います。



工夫している点

- **研究のサイクル全体を 1 か所に。** 検索 → 読解 → 比較 → 仮説 → 実験の流れを、5 つの画面でつなぎます。
- **Agent に目標を伝える操作。** ツールを順番に操作する代わりに、「最近の触覚 World Model の論文を探して」のような目標を伝えると、検索から登録まで Agent が進めます。
- **設計の判断を記録している。** ADR（技術選定の記録）と C4 モデルの構成図をリポジトリに残しています。LLM なしでも動くオプションもあります。

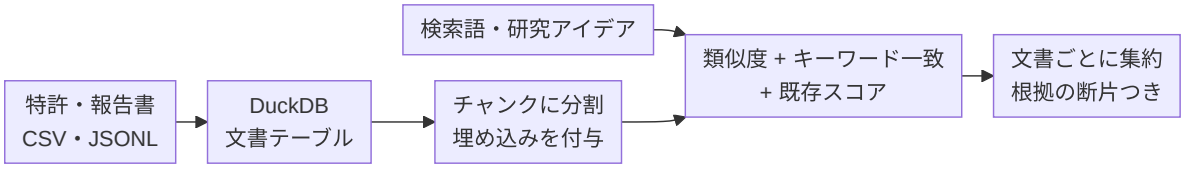
26. JAEA 特許・報告書検索（ローカル RAG）

DuckDB | RAG | Claude Code | Codex | ベクトル検索

GitHub: github.com/mutomasa/jaea_search_skill

成果物の概要

JAEA の特許と報告書を DuckDB に取り込み、Codex や Claude Code から `jaea-search "キーワード"` で関連資料を探すローカル RAG 検索システムです。関連の理由、技術分類、詳細 URL、PDF へのリンクを付けて返します。



工夫している点

- **外部 API にも学習済みモデルにも頼らない。** 文字 n-gram の埋め込みを自作し、日本語と英語が混ざった文書を検索できるようにしました。
- **根拠を示す。** どの本文の断片に反応したかを返すので、結果が正しいかを人が確かめられます。
- **検索語の長さで方法を変える。** 短い語は偶然の一致を避けるためキーワード一致を重視し、長い研究アイデアでは埋め込みの類似度も使います。
- **AI Agent のスキルとして使える。** Claude Code と Codex に使い方と返答形式を教える指示ファイルを付けています。

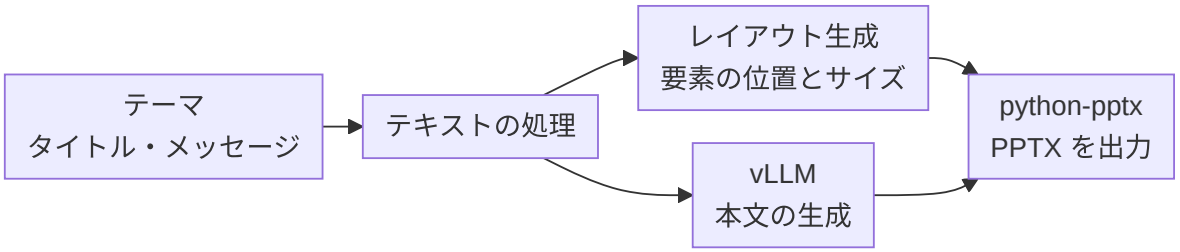
27. DLT スライド生成システム

vLLM | python-pptx | Streamlit

GitHub: github.com/mutomasa/dlt_genenrate_pptx

成果物の概要

テキストのテーマから、レイアウトの生成と LLM による本文の生成を組み合わせて、PowerPoint のスライドを作るシステムです。



工夫している点

- **レイアウトと内容を分けた。** 要素の配置と本文の生成を別の部品にし、それぞれを改善しやすくしています。
- **実装の範囲を正直に書いた。** レイアウト生成は現時点ではルールベースのテンプレートです。本来の DLT（Transformer ベースの深層学習）との違いと、置き換える計画を README に明記しています。

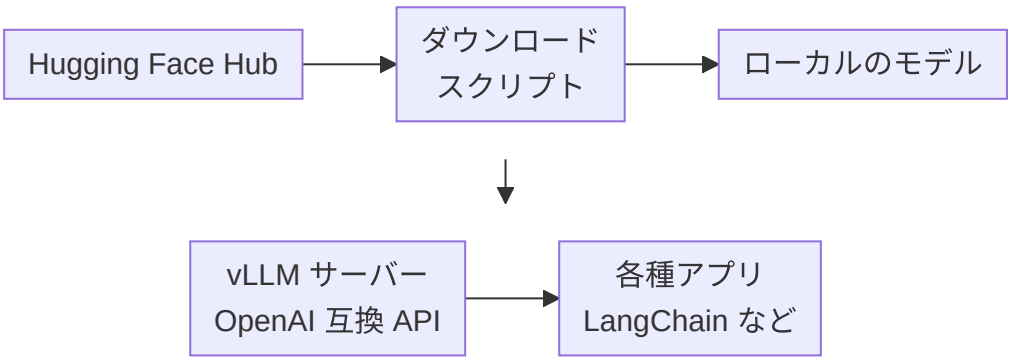
28. vLLM サーバーとモデルのダウンロード

vLLM | Hugging Face | OpenAI 互換 API | 量子化

GitHub: github.com/mutomasa/model_dl_vllm_server

成果物の概要

Hugging Face のモデルをダウンロードし、OpenAI 互換 API の vLLM サーバーを起動するツール一式です。8GB の GPU で動くモデルの一覧を付けています。



工夫している点

- **8GB の GPU で使える構成。** モデルごとのサイズとメモリ使用量を表にまとめ、量子化モデルも含めて選べるようにしました。
- **ほかのプロジェクトの土台。** OpenAI 互換 API にすることで、このポートフォリオの多くのアプリが同じ方法でローカル LLM を使えます。

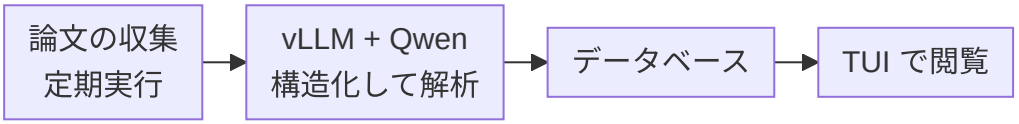
29. RoboPaper TUI（論文収集エージェント）

中断 | Textual | vLLM | Qwen2.5-7B | pytest

GitHub: github.com/mutomasa/paper_research_agent

成果物の概要

Robot Foundation Model 関連の論文を定期的に集め、ローカルの vLLM + Qwen で構造化してから、キーボード操作の TUI で読むためのツールです。開発を中断し、機能は Research OS に統合する予定です。



工夫している点

- **ローカル LLM で解析する。** OpenAI 互換 API を使うので、ローカルの vLLM でも、ほかのサーバーでも動きます。
- **コマンドを 2 つに分けた。** 収集・解析（research）と閲覧（tui）を分け、収集だけを定期実行できます。

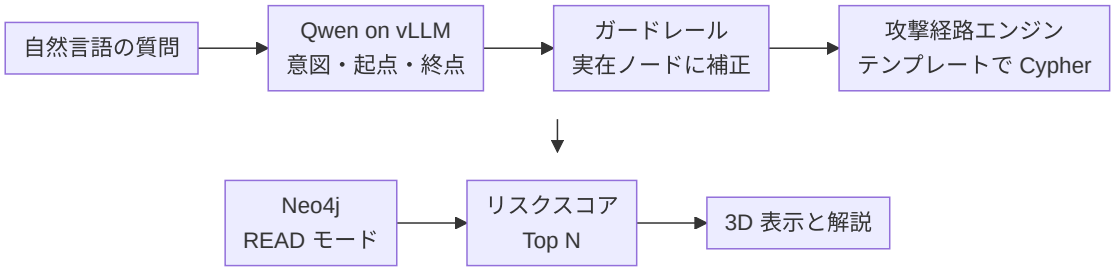
30. Security Attack Graph 3D

Neo4j | Cypher | vLLM | Qwen | FastAPI | Three.js | Streamlit

GitHub: github.com/mutomasa/security-attack-graph-3d

成果物の概要

企業システムの資産・脆弱性・権限・シークレット・データを知識グラフで表し、「どの経路で顧客データまで到達できるか」を自然言語で質問して、3D で確かめるプロトタイプです。脆弱性を「点」ではなく「経路」で捉え、対策の優先順位を経営層と現場が同じ図で合意できるようにします。



工夫している点

- **LLM に Cypher を書かせない。** LLM は意図・起点・終点を JSON で返すだけで、クエリはテンプレートから作ります。利用者の値はすべてパラメータで渡し、読み取り専用の検証をしてから READ モードで実行します。
- **説明が正確。** 経路の説明文はグラフから組み立てます。LLM の解説も、ノード名を経路から固定（guided decoding）して、存在しない名前を出させません。
- **スコアの内訳を見せる。** 脆弱性の深刻度 × 悪用のしやすさ、奪われる権限、データの重要度、経路の長さから順位を付け、内訳で「なぜ危険か」を説明できます。長い経路ほど点が上がる設計書の式の問題に気づき、修正しました。
- **LLM がなくても動く。** vLLM が落ちているときは、ルールベースの解析とテンプレートの説明に自動で切り替えます。
- **デモの録画を自動化した。** Playwright でアプリを自動操作して GIF を撮影し、字幕を付けています。

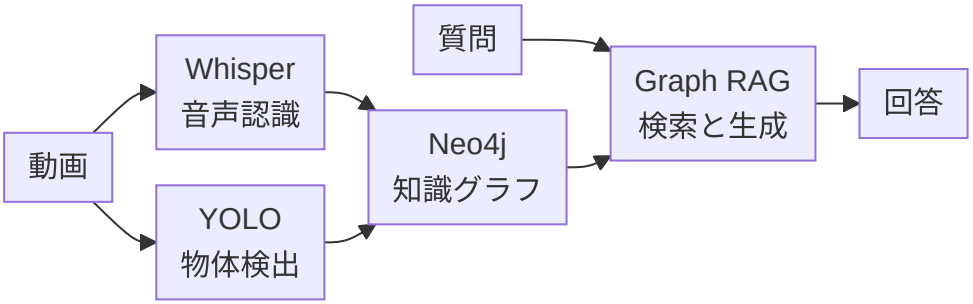
31. Graph RAG Video（動画分析）

Whisper | YOLO | Neo4j | Graph RAG | Streamlit

GitHub: github.com/mutomasa/graph_rag_video

成果物の概要

Whisper の音声認識と YOLO の物体検出で動画の内容を取り出し、Neo4j の知識グラフに入れて、Graph RAG で検索・回答する動画分析システムです。



工夫している点

- **音声と映像の両方を使う。** 話された内容（Whisper）と映っている物体（YOLO）を同じ知識グラフに入れ、動画の内容をまとめて検索できるようにしています。
- **関係で検索する。** ベクトル検索だけでなく、グラフの関係をたどって回答の根拠を集めます。

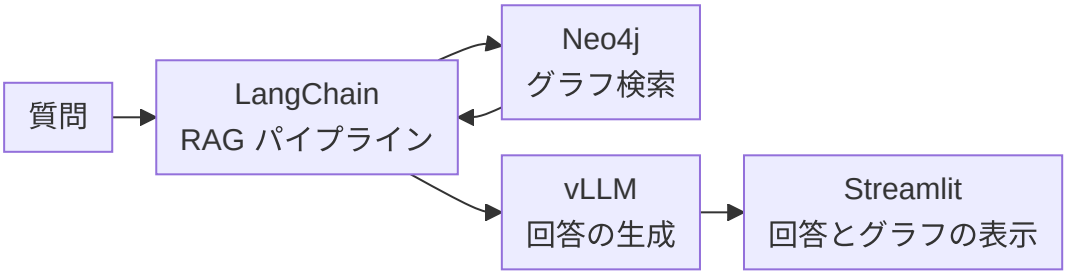
32. Graph RAG アプリ

Neo4j vLLM LangChain Streamlit

GitHub: github.com/mutomasa/graph_rag_app

成果物の概要

Neo4j のグラフデータベースと vLLM サーバーを LangChain でつないだ Graph RAG アプリです。グラフの可視化も付いています。



工夫している点

- **ローカルで完結する RAG。** 生成はローカルの vLLM で行うので、社内データを外部に送らずに試せます。
- **接続先を設定で切り替える。** Neo4j と vLLM の接続先は環境変数で設定します。

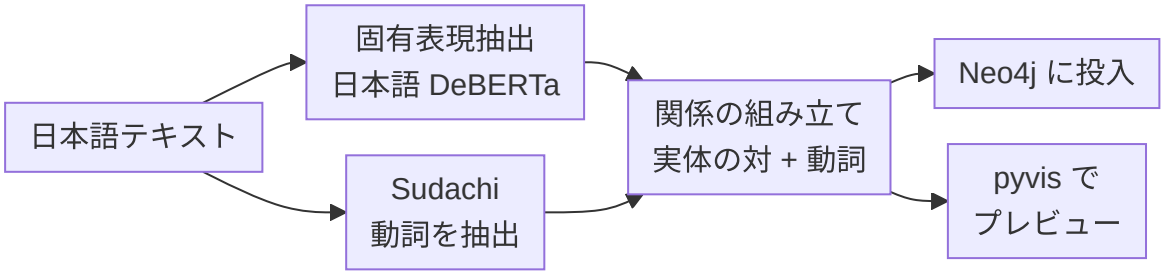
33. 日本語テキストからの知識グラフ構築

日本語 NER | DeBERTa | Sudachi | Neo4j | pyvis

GitHub: github.com/mutomasa/create_knowledge_graph_neo4j

成果物の概要

日本語のテキストから固有表現と簡単な関係を取り出し、Neo4j に知識グラフを作るツールです。CLI と、可視化付きの Streamlit アプリがあります。



工夫している点

- **投入前に確認できる。** 「抽出のみ」で結果とグラフを画面で確かめてから、Neo4j に入れられます。
- **軽い関係抽出。** 文中の実体の対に、Sudachi で取り出した動詞の原形を関係として付けます。動詞が見つからないときは「related_to」にします。

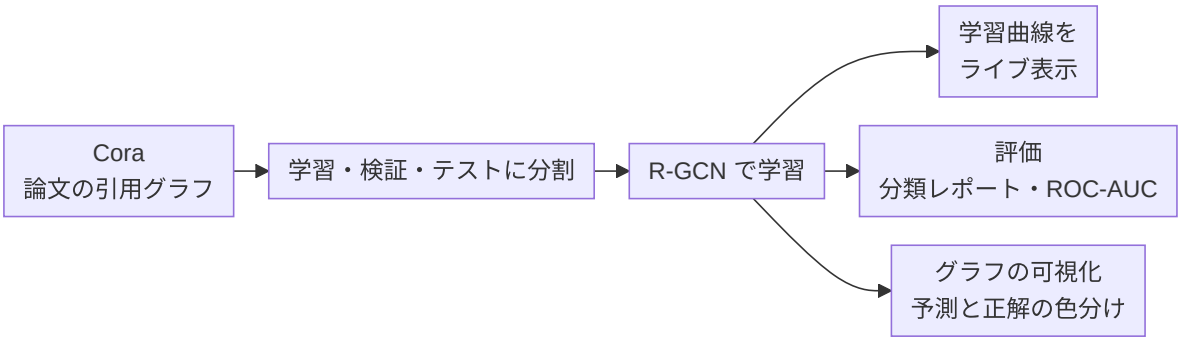
34. GNN 可視化アプリ（R-GCN on Cora）

PyTorch Geometric | R-GCN | Plotly | Streamlit

GitHub: github.com/mutomasa/gnn_visualization_app

成果物の概要

PyTorch Geometric の R-GCN を Cora データセットで学習・評価し、学習曲線とグラフの構造を可視化する Streamlit アプリです。



工夫している点

- **学習の進み具合が見える。** 損失と正解率を学習中にライブで更新します。
- **予測と正解を比べられる。** ノードの色を予測ラベルと正解ラベルで切り替え、間違えやすい部分をグラフ上で確かめられます。

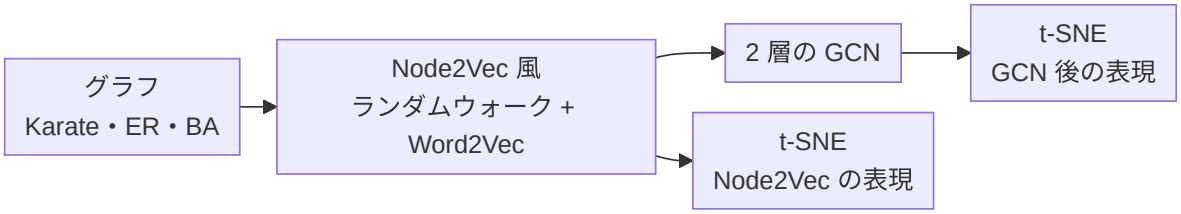
35. Node2Vec + GCN 可視化

Node2Vec | GCN | PyTorch Geometric | t-SNE

GitHub: github.com/mutomasa/node2vec_gcn_app

成果物の概要

NetworkX のグラフに Node2Vec で埋め込みを作り、2 層の GCN を通した後の表現と並べて、2 次元で比べる Streamlit アプリです。



工夫している点

- **左右に並べて比べる。** GCN が近くのノードの情報を集めて表現を滑らかにする様子を、前後で比べて理解できます。
- **グラフの種類を変えられる。** 構造の違う 3 種類のグラフで、埋め込みの違いを確かめられます。

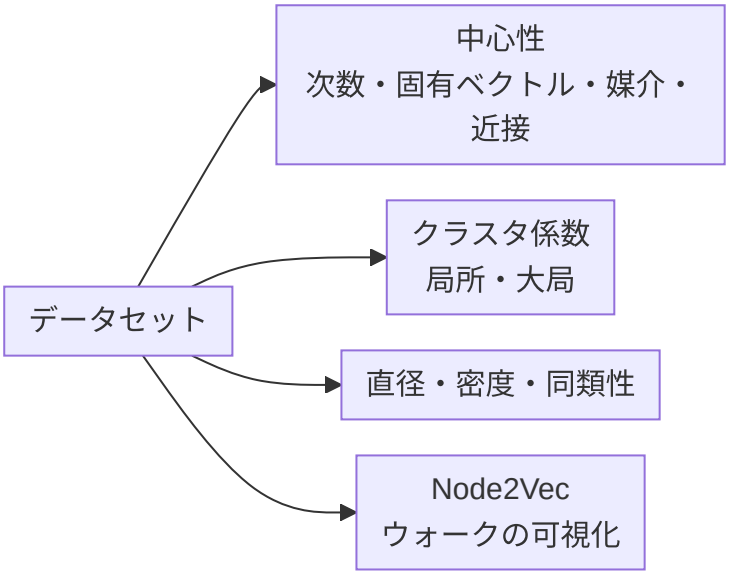
36. ネットワーク指標の可視化

PyTorch Geometric | NetworkX | Node2Vec | Streamlit

GitHub: github.com/mutomasa/network_indicator_visualization

成果物の概要

PyTorch Geometric のデータセットを使い、中心性・クラスタ係数・直径などのネットワーク指標と、Node2Vec の埋め込みを計算・可視化する Streamlit アプリです。



工夫している点

- **8 種類の指標をまとめて見られる。** どのノードが重要か、どれくらいまとまっているかを、指標ごとに比べられます。
- **探索の仕方を変えられる。** Node2Vec のパラメータで、近くを探るか遠くへ行くかを変え、ウォークの様子をその場で表示します。

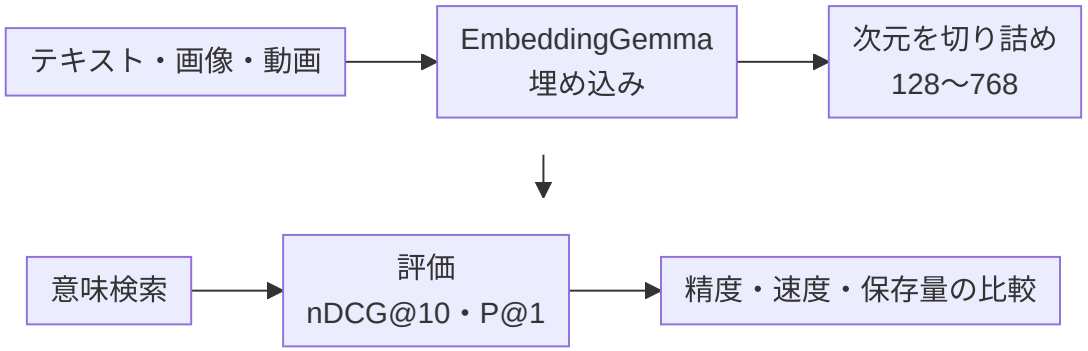
37. EmbeddingGemma Multimodal Lab

EmbeddingGemma | Matryoshka 埋め込み | マルチモーダル検索 | Streamlit

GitHub: github.com/mutomasa/embeddinggemma-multimodal-lab

成果物の概要

`google/embeddinggemma-2` で、テキスト・画像・動画の意味検索を行い、Matryoshka の次元（128 / 256 / 512 / 768）ごとの精度・速度・保存量の違いをローカルで比べるデモです。正解ラベル付きの評価セットと、人が付けたラベルとの照合で、数値で比較できます。



工夫している点

- **見せるだけでなく測る。** 検索結果を表示するだけでなく、nDCG@10 などで次元ごとの精度を比べます。テキストでは 0.905〜0.917、画像では 0.948〜0.965 でした。
- **コストも測る。** 1 ベクトルの保存量（512B〜3KB）、クエリのエンコード時間（約 19〜22ms）、検索時間（約 0.03〜0.04ms）を記録しています。
- **結果の限界を正直に書く。** サンプルが小さく次元間の差は誤差の範囲であることや、合成動画で結果が偏る問題を、既知の問題として README に記録しています。

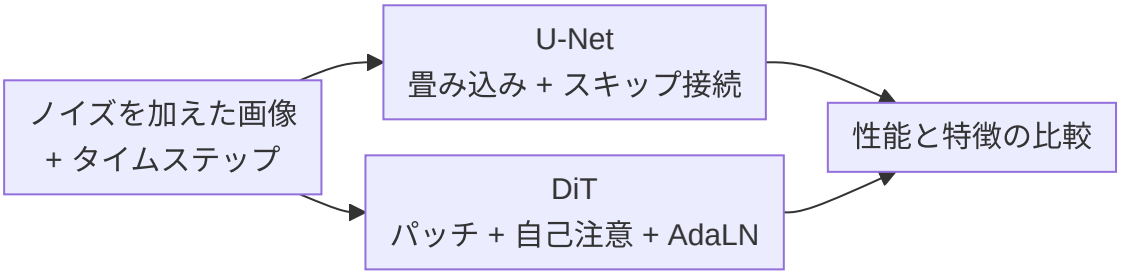
38. Diffusion Transformer vs U-Net

拡散モデル DiT U-Net PyTorch

GitHub: github.com/mutomasa/cnn_unet_diffusion_transformer_app

成果物の概要

拡散モデルの中心となるネットワークについて、CNN ベースの U-Net と Transformer ベースの DiT（Diffusion Transformer）を実装し、構造と性能の違いを可視化して比べるプロジェクトです。



工夫している点

- **同じ課題で 2 つの構造を比べる。** 局所的な特徴に強い U-Net と、全体の関係を捉える DiT の違いを、同じ拡散モデルの中で確かめます。
- **条件付けの仕組みも実装した。** DiT では、タイムステップとクラスラベルで正規化を調整する AdaLN を実装しています。

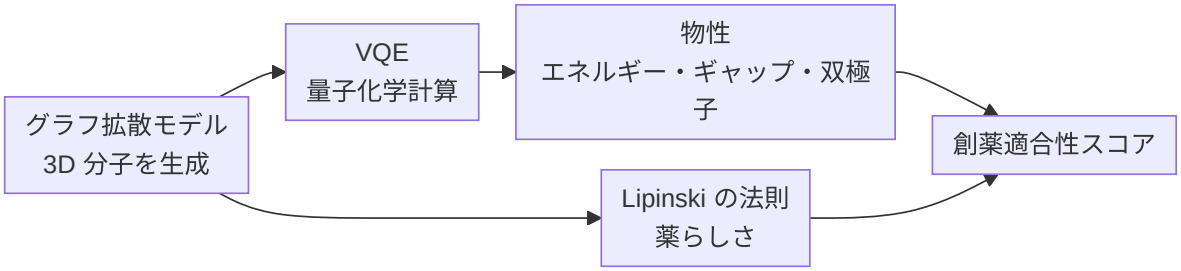
39. AI 駆動創薬 × 量子コンピュータ デモ

グラフ拡散モデル | E(3) 等変 | VQE | 量子化学 | Streamlit

GitHub: github.com/mutomasa/graph-diffusion-quantum-molecular

成果物の概要

E(3) 等変なグラフ拡散モデルで 3D の分子構造を生成し、量子計算（VQE）で基底状態エネルギーや HOMO-LUMO ギャップを求める創薬支援のデモです。



工夫している点

- **生成と評価を 1 つの流れに。** 分子の生成、量子計算による物性の予測、薬らしさの評価を組み合わせ、候補の総合スコアを出します。
- **生成の過程を見せる。** 分子ができあがっていく過程を 3D のアニメーションで表示します。

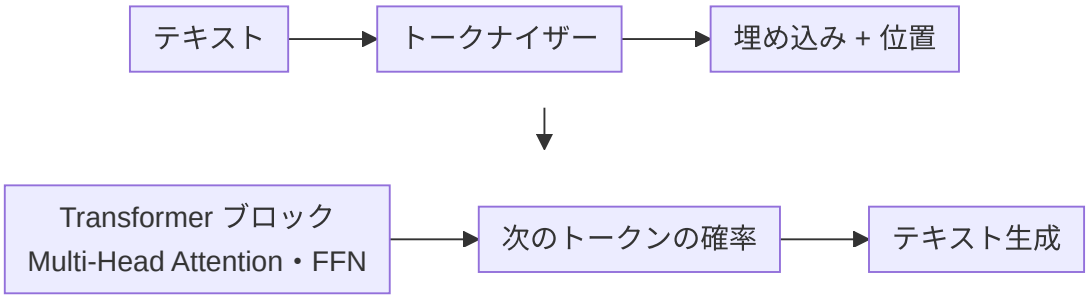
40. Simple GPT（学習用）

PyTorch | Transformer | Hugging Face tokenizers

GitHub: github.com/mutomasa/simple_gpt_for_learning

成果物の概要

GPT の仕組みを理解するために、PyTorch で書いたシンプルな GPT です。Hugging Face のトークナイザーを使い、学習とテキスト生成の両方ができます。



工夫している点

- **読んで分かる大きさ。** Multi-Head Attention、Feed Forward、正規化、位置埋め込みを、理解しやすい形で実装しています。
- **学習から生成まで。** 1 つのスクリプトで学習と生成のモードを切り替えられます。

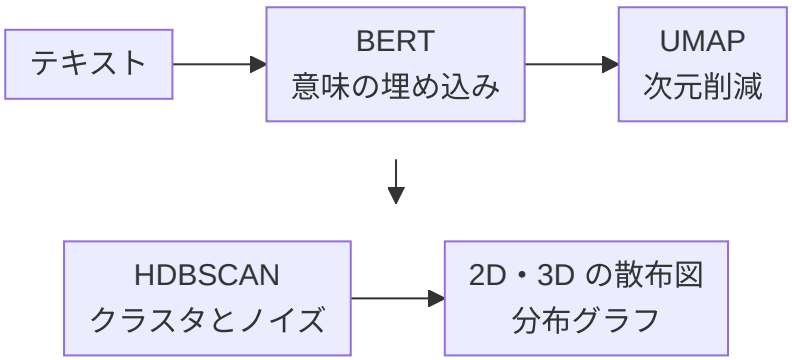
41. BERT + UMAP + HDBSCAN テキストクラスタリング

Sentence Transformers | UMAP | HDBSCAN | Streamlit

GitHub: github.com/mutomasa/bert_umap_hdbscan_clustering

成果物の概要

BERT の埋め込み、UMAP の次元削減、HDBSCAN の密度ベースクラスタリングを組み合わせ、テキストを自動で分類して可視化する Streamlit アプリです。



工夫している点

- **クラスタ数を決めなくてよい。** HDBSCAN でクラスタ数を自動で決め、どこにも属さないテキストをノイズとして分けます。
- **パラメータを調整できる。** UMAP の近傍数や最小距離、モデルの種類を変えて、分類の変化を確かめられます。

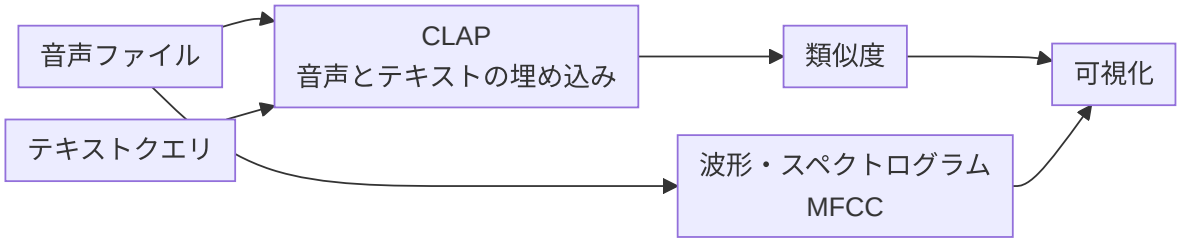
42. CLAP 音声とテキストの理解

CLAP librosa Plotly Streamlit

GitHub: github.com/mutomasa/clap_app

成果物の概要

CLAP（Contrastive Language-Audio Pretraining）で、音声ファイルとテキストのクエリの類似度を計算し、波形・スペクトログラム・MFCC と合わせて可視化する Streamlit アプリです。



工夫している点

- **クエリを提案する。** 音声のカテゴリに合わせて、試すとよいテキストのクエリを提案します。
- **テスト用の音声を用意した。** カテゴリ別のサンプル音声と合成音声の生成機能で、手元に音声がなくても試せます。

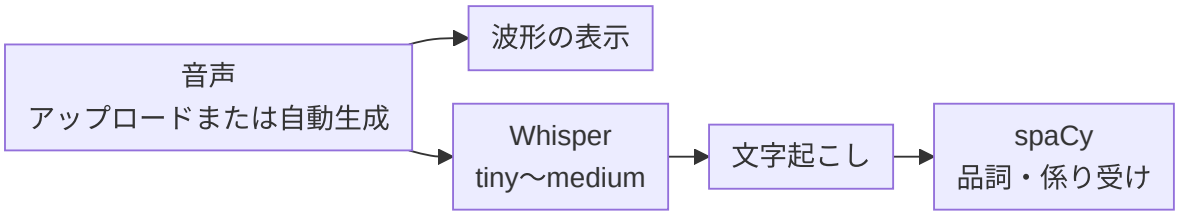
43. Whisper 日本語音声認識デモ

Whisper spaCy gTTS Streamlit

GitHub: github.com/mutomasa/whisper_app

成果物の概要

日本語の音声を Whisper で文字起こしし、波形の表示と spaCy による形態素解析まで行うデモアプリです。



工夫している点

- **音声がなくても試せる。** gTTS でデモ用の日本語音声を自動で作ります。
- **モデルの大きさを選べる。** tiny から medium まで切り替えて、精度と速度の違いを確かめられます。

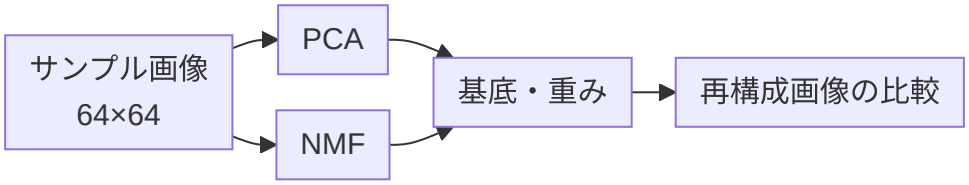
44. PCA と NMF の可視化

scikit-learn | Plotly | mypy | pytest

GitHub: github.com/mutomasa/pca_nmf_app

成果物の概要

グレースケール画像に対する主成分分析（PCA）と非負値行列因子分解（NMF）を、基底と再構成画像を見ながら比べて学ぶ Streamlit アプリです。



工夫している点

- **2 つの手法の違いが見える。** 画像を基底の線形和として表す方法の違い（負の値を許すか）を、基底の画像と再構成で比べられます。
- **品質管理の仕組み。** ruff、mypy、pytest で静的解析・型チェック・単体テストを行っています。

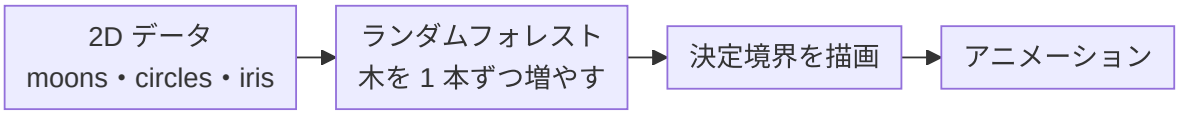
45. ランダムフォレストの決定境界

scikit-learn Plotly Streamlit

GitHub: github.com/mutomasa/random_forest_app

成果物の概要

ランダムフォレストの決定境界が、木の本数を増やすにつれてどう変わるかをアニメーションで見る Streamlit アプリです。



工夫している点

- **アンサンブルの効果を直感的に。** 木が増えるほど予測が安定していく様子を、アニメーションで観察できます。

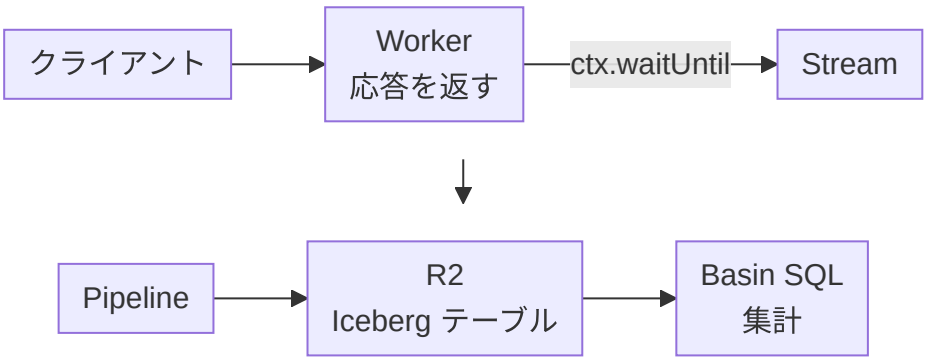
46. Cloudflare Basin + Iceberg

Cloudflare Workers | Pipelines | R2 | Apache Iceberg | Vitest

GitHub: github.com/mutomasa/cloudflare_basin_iceberg

成果物の概要

Cloudflare Workers で受けたリクエストをイベントとして Pipelines に送り、R2 上の Apache Iceberg テーブルに貯めて、SQL で集計するサンプルです。



工夫している点

- **応答を遅くしない。** イベントの送信は `ctx.waitUntil` で応答の後に行い、利用者への応答時間に影響させません。
- **サーバーなしのデータ基盤。** サーバーを持たずに、ログの収集から分析用テーブル（Iceberg・Parquet）までを作れます。
- **構成を文書にした。** 各部品の役割とデータの流れを図と文章で説明し、Vitest のテストも付けています。

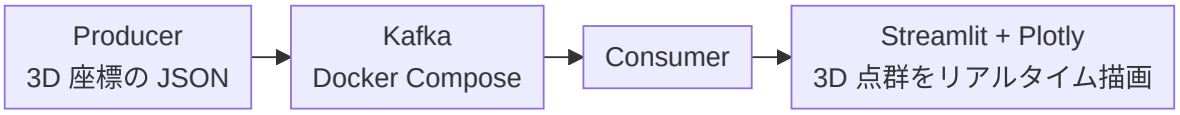
47. Kafka 3D ストリーミング可視化

Kafka | Docker Compose | Plotly | Streamlit

GitHub: github.com/mutomasa/kafka_visualize_stream_app

成果物の概要

ローカルの Kafka（Docker Compose）に 3D 座標の JSON ストリームを流し、Streamlit と Plotly でリアルタイムに描画するアプリです。



工夫している点

- **すぐに動かせる。** Kafka と Zookeeper を Docker Compose で立ち上げ、テスト用のデータを送る Producer も付けています。
- **表示を調整できる。** 表示する点の数とポーリング間隔を画面から変えられます。

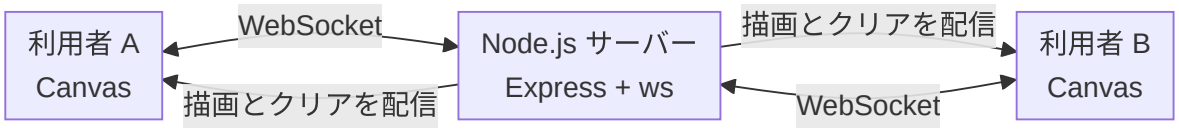
48. リアルタイム共有ホワイトボード

TypeScript Node.js Express WebSocket Canvas

GitHub: github.com/mutomasa/websocket_whiteboard_app

成果物の概要

TypeScript・Express・WebSocket で作った、複数の人が同時に描けるリアルタイムの共有ホワイトボードです。



工夫している点

- **切れても戻る。** 接続が切れると自動で再接続し、接続の状態を画面に表示します。
- **リモート開発でも動く。** VS Code のポートフォワードを使うリモート環境での手順を用意しました。

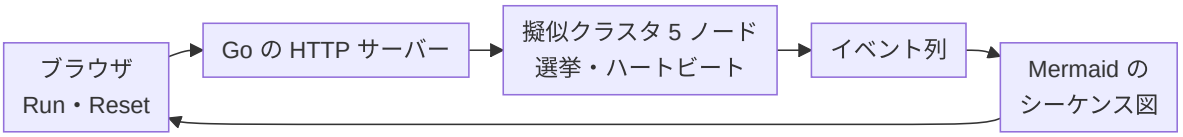
49. Raft の可視化（Go）

Go Raft Mermaid

GitHub: github.com/mutomasa/go_visualize_raft_app

成果物の概要

分散合意アルゴリズム Raft のリーダー選出とハートビートを擬似的に生成し、Mermaid のシーケンス図としてブラウザに表示する Go の学習用デモです。



工夫している点

- **メッセージの流れが見える。** 選挙からリーダーの確定、ハートビートまでのやり取りを、シーケンス図で順を追って確かめられます。
- **範囲を明記した。** 学習用の擬似実装であり、実運用の合意の要件を満たさないことを README に書いています。

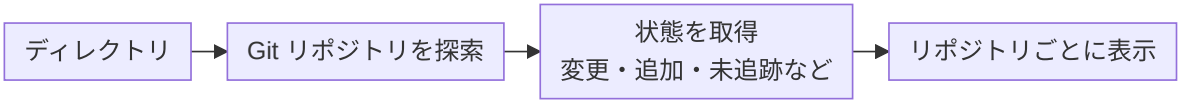
50. Git Uncommit Checker

Go CLI Git

GitHub: github.com/mutomasa/git_uncommit_checker

成果物の概要

指定したディレクトリの下にある Git リポジトリをすべて探し、コミットしていないファイルを一覧にする Go の CLI です。



工夫している点

- **状態を見分けやすく。** 変更・追加・削除・未追跡など 8 種類の状態を、絵文字で区別して表示します。
- **多数のリポジトリを一度に確認。** 作業の終わりにコミット漏れがないかをまとめて確かめられます。

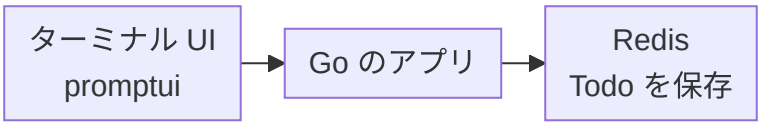
51. ターミナル Todo アプリ

Go | Redis | Docker Compose | promptui

GitHub: github.com/mutomasa/go_todo_db_terminal

成果物の概要

Go と Redis で作ったターミナルの Todo アプリです。Docker Compose で動きます。



工夫している点

- **環境構築が要らない。** アプリと Redis を Docker Compose でまとめて起動できます。
- **対話的な操作。** 追加・一覧・完了の切り替え・削除を、ターミナルのメニューで操作できます。

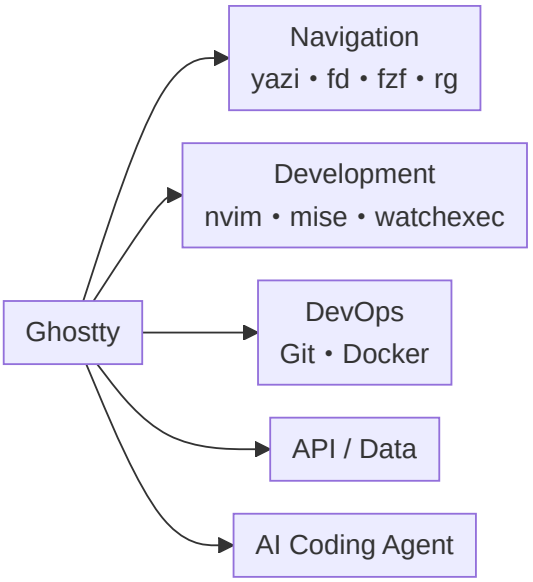
52. Ghostty AI-Native 開発環境

Ghostty Neovim yazi fzf mise AI Coding Agent

GitHub: github.com/mutomasa/my_dev_environment

成果物の概要

ターミナル Ghostty を中心に、ファイル操作・編集・Git・Docker・API・データベース・AI Coding Agent までをターミナルで完結させる開発環境の構成をまとめたものです。



工夫している点

- **ツール同士の連携を表にした。** 各ツールの役割・用途・ほかのツールとの連携を一覧にし、環境全体を説明できるようにしています。
- **GUI への依存を減らす。** ターミナルを開発環境全体の入口にし、AI Coding Agent との作業もそこで完結させます。

53. Engineering Research Portfolio（このサイト）

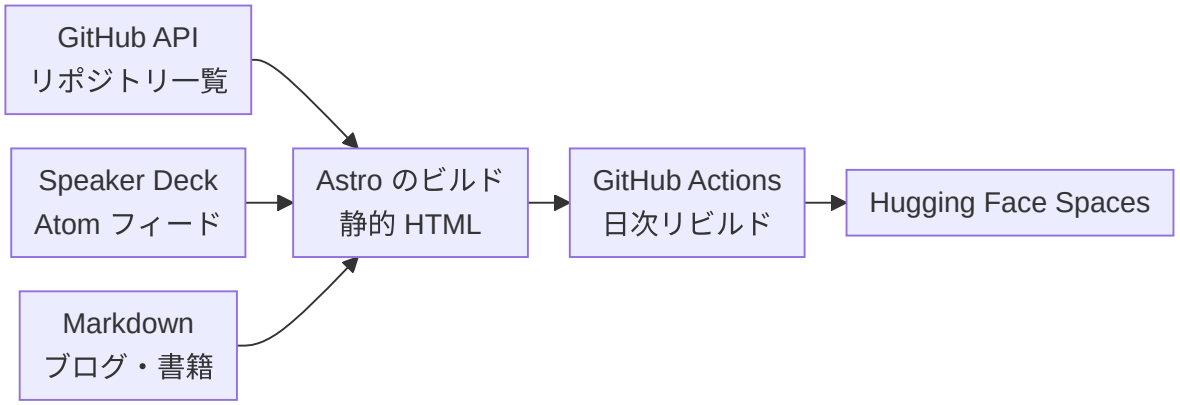
Astro | GitHub Actions | Hugging Face Spaces | GitHub API

GitHub: github.com/mutomasa/engineering-research-portfolio

サイト: huggingface.co/spaces/mutomasa/engineering-research-portfolio

成果物の概要

研究ポートフォリオのサイトです。Astro の静的出力で作り、GitHub Actions から Hugging Face Spaces にデプロイしています。プロジェクト、研究、発表資料、ブログ、技術書籍（アーキテクチャ入門）を載せています。



工夫している点

- **サーバーを持たない。** ビルド時に GitHub と Speaker Deck から情報を取得して静的 HTML に埋め込み、毎日リビルドして最新に保ちます。
- **取得に失敗しても壊れない。** API の取得に失敗したときは、各サービスのページへのリンクを表示します。
- **日本語と英語に対応。** 言語の切り替え、数式（KaTeX）、Mermaid の図に対応しています。